Estructuras

Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

15 / 2026

r33

jagged array

¡A la terminal!




¿Preguntas?

Estructuras de información (structs)

UNRN Universidad Nacional de Río Negro

Declaración

```
struct identificador
{
   tipo identificador_miembro_1;
   tipo identificador_miembro_2;
…
   tipo identificador_miembro_n;
};
```

Declaración

```
struct fraccion
{
   int numerador;
   int denominador;
};
```

Data este `struct`

```
struct persona {
  int edad;
};

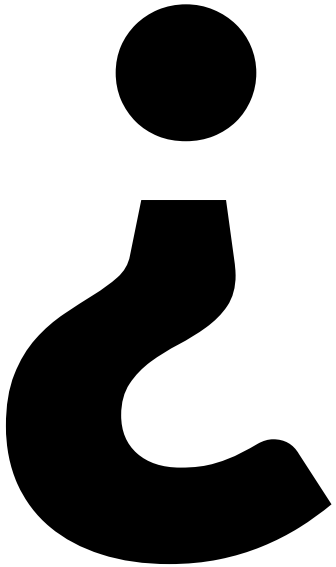
void imprimir_edad(struct persona *p) {
    printf("La edad de la persona es %d\n", p->edad);
}
```

La creación de dos variables

```
int main() {
  struct persona p1;
  struct persona p2;
  p1.edad = 40;
  p2.edad = 19;
  printf("La edad de la persona es %d\n", p.edad);
  imprimir_edad(&p);

  return 0;
}
```

da dos `edad` separados, dentro de `p1` y `p2` respectivamente



Como se usan



Inicialización

```
struct fraccion f2 = {1, 4};
struct fraccion f2 = {0};
```

Inicialización

(más explícita)

```
struct fraccion f3 = { .denominador = 1, .numerador = 3 };
```

Uso y acceso a miembros

```
struct fraccion f1;
f1.numerador = 10;
f1.denominador = 10;
```



Otro ejemplo

```
struct Persona
{
    char nombre[50];
    int edad;
    float altura;
};
```

```
struct Persona persona1 = {"Juan",
30, 1.75};

struct Persona persona2;
strcpy(persona2.nombre, "Ana");
persona2.edad = 25;
persona2.altura = 1.60;
```

Se pueden anidar!

```
struct Fecha
{
  int dia;
  int mes;
  int año;
};
struct Persona
{
    char nombre[50];
    int edad;
    struct Fecha nacimiento;
};
```

```
struct Persona persona1 = {"Juan",
30, 1.75};

struct Persona persona2;
strcpy(persona2.nombre, "Ana");
persona2.edad = 25;
persona2.nacimiento = {1, 2, 2000};
persona2.nacimiento.dia = 2;
```


¿Preguntas?

Uso en funciones

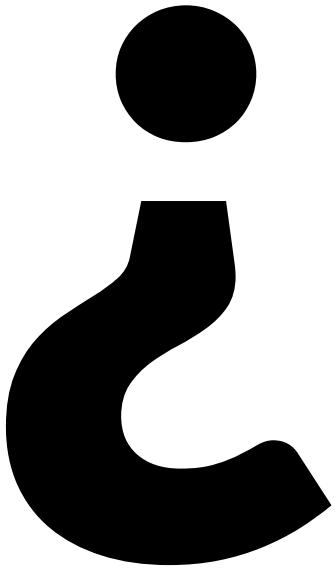
```
struct fraccion suma_fraccion(struct fraccion frac, int numero);
```



Uso en funciones

```
struct Mensaje
{
    char texto[50];
};
```

```
void imprime(struct Mensaje)
{
  …
}
```



Como pasa el `struct` a la función



¡Por copia!

{ Cuidado con las estructuras que pasan }


¿Preguntas?

Operaciones con structs

UNRN Universidad Nacional de Río Negro

Copia de estructuras

```
struct fraccion f1 = {1, 4};
struct fraccion f2 = f1;
```

¿Comparación directa?

```
f2 == f1
```

¿Comparación directa?

```
f2 == f1
```

¡No funciona!

Pero, miembro a miembro sí

```
f1.denominador > f2.denominador
```

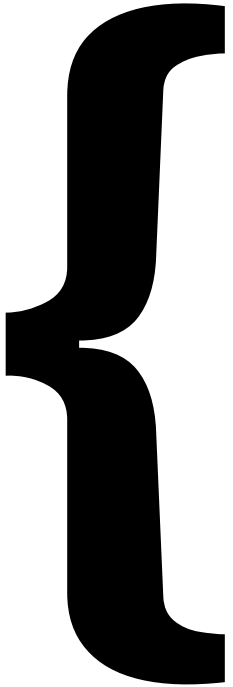

¿Preguntas?

Con esto en mente...

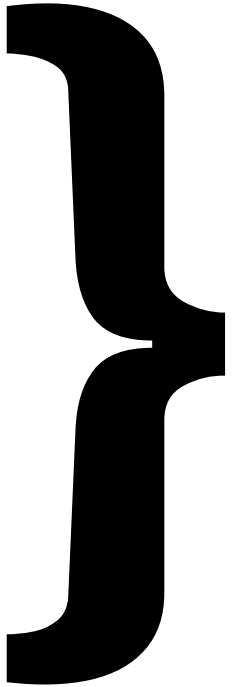
UNRN Universidad Nacional de Río Negro

```
struct division {
    int cociente;
    int resto;
};


struct division division_lenta(int dividendo, int divisor);
```



Al crear un `struct`...



No superpongan la entrada y salida

```
struct division {
    int dividendo;
    int divisor;
    int cociente;
    int resto;
};

struct division division_lenta(struct division argumentos);
```


¿Preguntas?

Sobrenombres (alias)

UNRN Universidad Nacional de Río Negro



```
typedef struct fraccion fraccion_t;

fraccion_t f3;
```

Declaración de un `struct` anónimo con alias

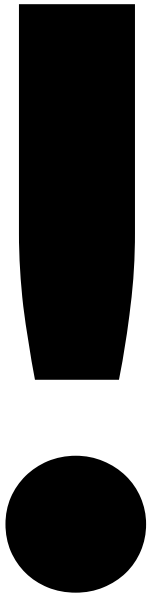
```
typedef struct
{
    int cociente;
    int resto;
} division_t;
```

Utilizá `typedef` para definir tipos de `estructuras` (y usando el sufijo `_t`)


¿Preguntas?



Podemos crear arreglos



```
fraccion_t multiple[MAX];
```

Punteros

UNRN Universidad Nacional de Río Negro

Nos falta memoria dinámica para que sea útil

```
fraccion_t fraccion = {2, 3};

fraccion_t *ptr_fraccion = &fraccion;
```

Nos falta memoria dinámica para que sea útil

```
fraccion_t fraccion = {2, 3};

fraccion_t *ptr_fraccion = &fraccion;

…

(*ptr_fraccion).denominador
```

Para desreferenciar un puntero a struct

O con la notación compacta

```
fraccion_t fraccion = {2, 3};

fraccion_t *ptr_fraccion = &fraccion;

…

ptr_fraccion->.denominador == (*ptr_fraccion).denominador
```

Para desreferenciar un puntero a struct


¿Preguntas?

punteros a estructuras

UNRN Universidad Nacional de Río Negro

Dada esta estructura, ¿cómo implementamos ‘crear’?

```
struct persona
{
    char nombre*;
    int edad;
}
```

```
struct persona* crear(char nombre[], int edad)
```

¡A la terminal!



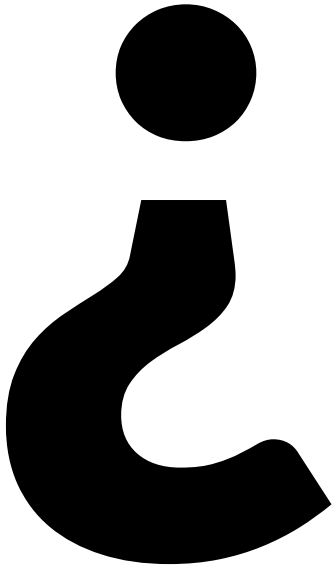
Resuelto

```
persona_t* crear(char nombre[], int edad)
{
    persona_t* nuevo = malloc(sizeof(persona_t));
    if (nuevo != NULL)
    {
        nuevo->nombre = malloc(sizeof(char)*strlen(nombre)+1);
        strcpy(nuevo->nombre, nombre);
        nuevo->edad = edad;
    }
    return nuevo;
}
```

¿Qué son estas dos cosas?

```
persona_t* crear(char nombre[], int edad)
{
    persona_t* nuevo = malloc(sizeof(persona_t));
    if (nuevo != NULL)
    {
        nuevo->nombre = malloc(sizeof(char)*strlen(nombre)+1);
        strcpy(nuevo->nombre, nombre);
        nuevo->edad = edad;
    }
    return nuevo;
}
```





Que es la

->



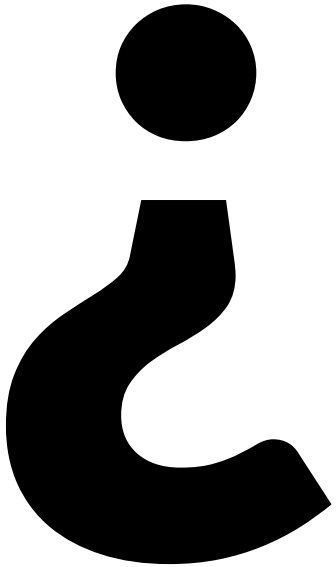
```
t_persona* usr;
```

```
usr->nombre
```

es igual a

```
(*usr).nombre
```

La flecha desreferencia el puntero



Que falta ahora



Para liberar al mismo nivel

```
void destruir(struct persona* persona)
```

¡A la terminal!

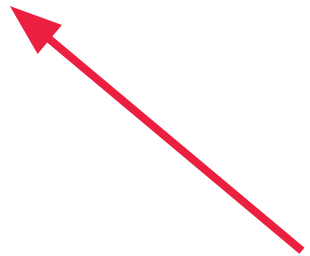


¡De dentro hacia afuera!

```
void destruir (persona_t* persona)
{
    free(persona->nombre);
    free(personal);
}
```

¿Por qué en este orden?

```
void destruir (persona_t* persona)
{
    free(persona->nombre);
    free(personal);
}
```




¿Preguntas?

punteros en estructuras

UNRN Universidad Nacional de Río Negro

Dada la siguiente estructura

```
typedef struct{
    int capacidad;
    char* cadena;
}cadena_t;
```

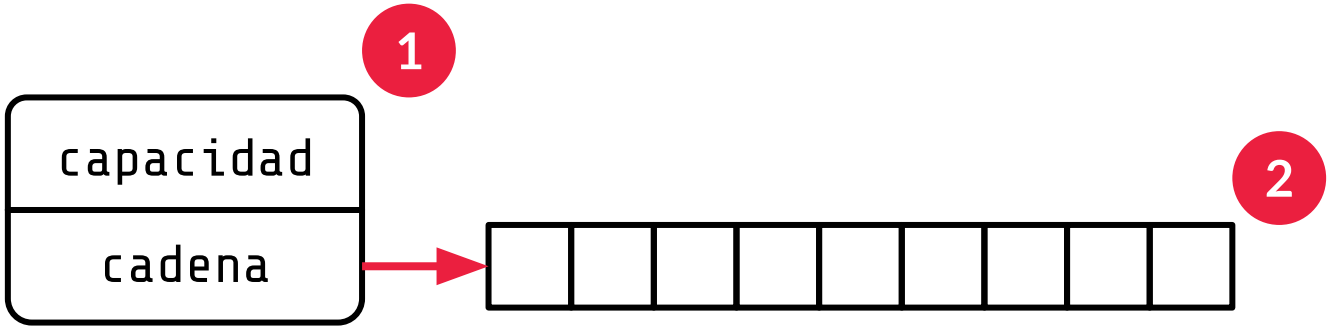
```
cadena_t* crear(int capacidad)
```



¡A la terminal!



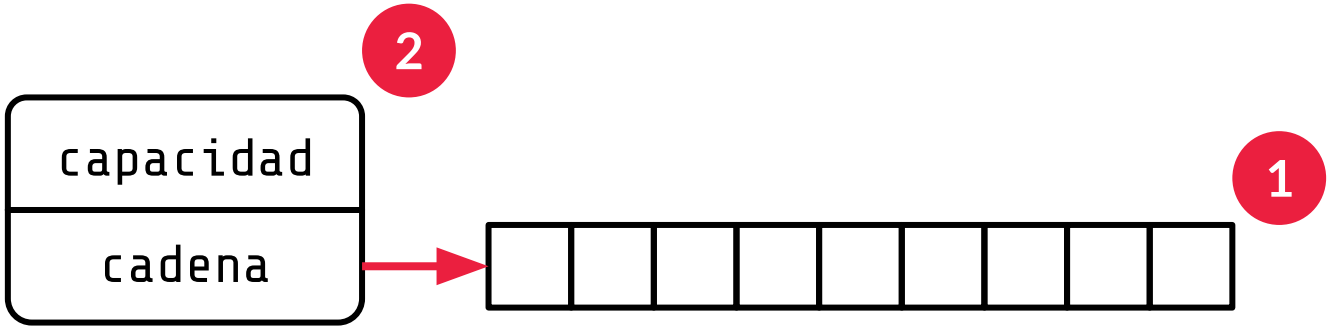
```
cadena_t* crear(int capacidad)
```



Con un `malloc` para cada parte

{ Y para liberar al mismo nivel }

```
void destruir(cadena_t* cadena)
```



Al revés que se crea

La documentación para `crear`

```
/**
 * Reserva la memoria para una cadena segura
 * @param capacidad el tamaño el arreglo para
 *                  almacenar los caracteres
 * @returns una cadena segura creada dinámicamente
 * @pre capacidad es mayor a 1
 * @post La estructura y arreglo de caracteres creados
 *          dinámicamente
 * @post El arreglo se garantiza en \0 en toda su extensión
 * @nota Usar la función 'destruir' para liberar la memoria
 */
cadena_t* crear(int capacidad);
```


¿Preguntas?

Estructuras auto-referenciales

UNRN Universidad Nacional de Río Negro

```c
typedef struct spider {
    // Puntero autoreferencial:
    //            apunta a otra instancia de 'struct s'.
    struct spider *finger;
    // Opcionalmente, se podrían añadir
    //            otros campos de datos aca.
} spiderman_t; // El alias (typedef) simplifica el uso posterior
```

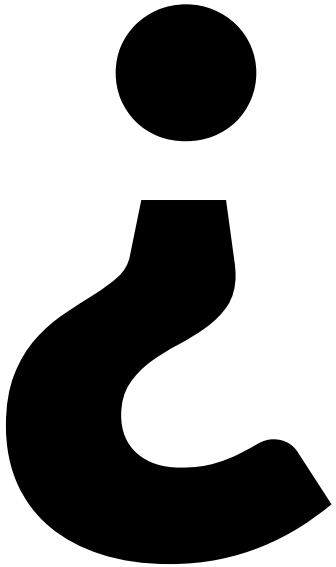


¿Puede una estructura apuntarse a sí misma?

gracias Jean

{ Le vamos a dar bastante uso }


¿Preguntas?



Que espacio ocupa un `struct` en memoria



Lógicamente, la suma de sus miembros

Fraccion	numerador	denominador

Sin importar que sea

Fecha	dia	mes	año

Persona	char[50] Nombre	edad	Fecha nacimiento		
			dia	mes	año


¿Preguntas?

pero....

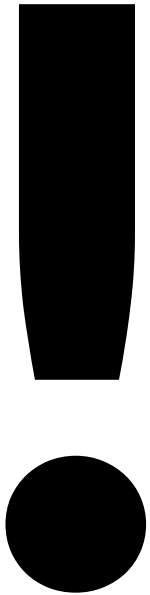
las

uniones

UNRN Universidad Nacional de **Río Negro**



Hacen lo opuesto

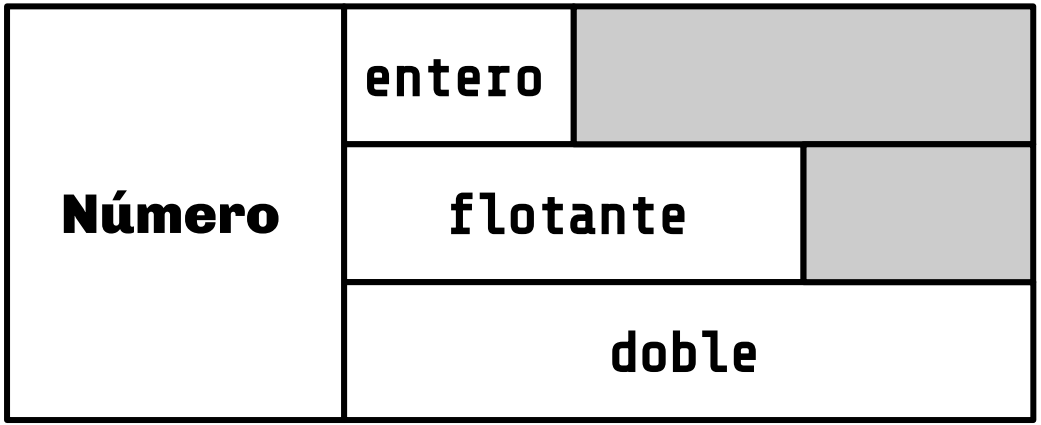


Declaración

```
union identificador {
   tipo identificador_miembro_1;
   tipo identificador_miembro_2;
…
   tipo identificador_miembro_n;
};
```

El tamaño es del más grande, el `double` en este caso

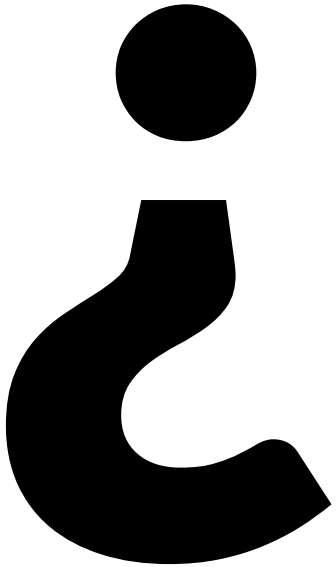
```
union Numero
{
    int entero;
    float flotante;
    double doble;
};
```



Cambian la interpretación de una parte de la memoria

Permite recibir una "figura" sin importar cuál sea

```
union figura {
    struct {
        float radio;
    } circulo;
    struct {
        float base, altura;
    } rectangulo;
    struct {
        float lado1, lado2, lado3;
    } triangulo;
};
```

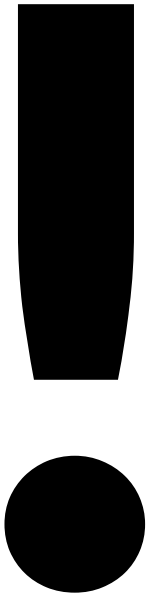


Podemos saber que vino



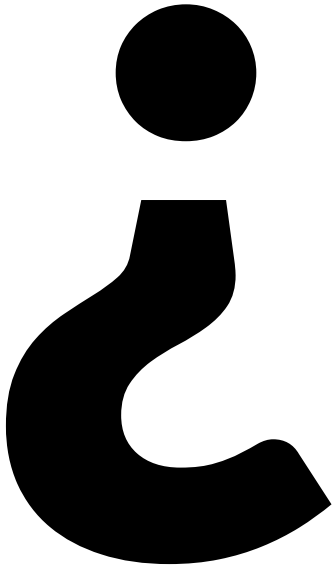


nope



Por eso se los combina con `structs`

```
struct figura_geometrica
{
    int tipo; // 0: círculo, 1: rectángulo, 2: triángulo
    union figura datos;
};
```



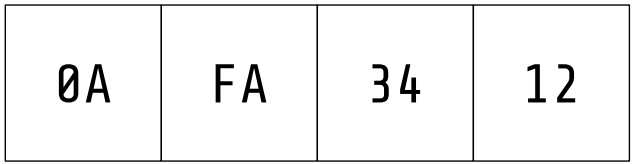
Para que se usan



Podemos acceder fácilmente a cada byte en un word

```
union ColorRGBA
{
    struct bytes{
        uint8_t rojo;
        uint8_t verde;
        uint8_t azul;
        uint8_t transparencia;
    };
    uint32_t hexadecimal;
};
```

Y podemos




```
union ColorRGBA color = {0};
color.bytes.rojo = 0xA;
color.bytes.verde = 0xFA;
color.bytes.azul = 0x34;
color.bytes.transparencia = 0x12;

color.hexadecimal => 0x0AFA3412;
```

{ Se usan mucho en µcontroladores }


¿Preguntas?

enumeraciones

Constantes numéricas agrupadas

Sintaxis báse

```
enum nombre_enum {
    constante1,  // Este, es el valor 'implícito' 0
    constante2,  // y sucesivamente 1
    constante3,  // 2
    // ...
};
```

Es a efectos prácticos, un `int`

Enumeración implícita por default

```
enum dias_semana {
    LUNES,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES,
    SABADO,
    DOMINGO
};
```

```
enum dias_semana hoy = MARTES;
if (hoy == VIERNES)
{
  printf("¡Es finde!");
}

int dia_numerico = DOMINGO;
```

Enumeración implícita

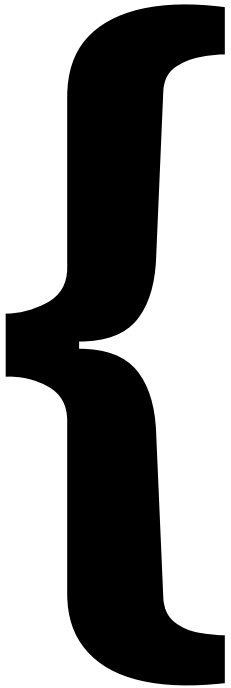
```
enum dias_semana {
    LUNES = 1,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES,
    SABADO,
    DOMINGO
};
```

```
enum dias_semana hoy = MARTES;
if (hoy == VIERNES)
{
printf("¡Es fin de semana!\n");
}

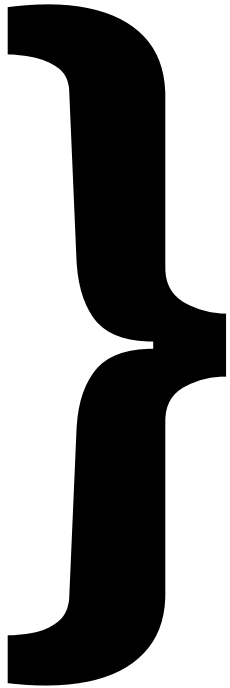
int dia_numerico = DOMINGO;
 // DOMINGO es ahora 7
```

Enumeración explícita

```
enum {
  CERO = 0,
  UNO = 10,
  DOS = 20
};
```



Al igual que los `struct`, al utilizarlo va entero



Puede ir como retorno de función

```
enum retornos{
    EXITO,
    ERROR_ARCHIVO_NO_ENCONTRADO,
    ERROR_MEMORIA_INSUFICIENTE,
    // ... otros posibles errores
};

enum retornos abrir_archivo(const char *nombre_archivo);
```

Y utilizar `typedef` para que sea más compacto

```
typedef enum {
    EXITO,
    ERROR_ARCHIVO_NO_ENCONTRADO,
    ERROR_MEMORIA_INSUFICIENTE,
    // ... otros posibles errores
} estado_archivo_t;

estado_archivo_t abrir_archivo(const char *nombre_archivo);
```

Con una `enum`eración anónima con alias

Usos interesante

Para representar de forma compacta bitmasks*

```
typedef enum {
    // Binario: 0001
    PERMISO_LECTURA   = 1 << 0,  // O simplemente 1
    // Binario: 0010
    PERMISO_ESCRITURA = 1 << 1,  // O simplemente 2
    // Binario: 0100
    PERMISO_EJECUCION = 1 << 2,  // O simplemente 4
    // Binario: 1000.
    PERMISO_AUDITORIA = 1 << 3   // O simplemente 8
} permiso_t;
```

Combinado con `structs`

```
typedef struct
{
    int z;
    enum Estado { SOLIDO, LIQUIDO, GAS, PLASMA } estado;
} elemento_t;

elemento_t oxigeno = { 8, GAS };
```

Con declaración y uso en la misma línea


¿Preguntas?

Problemas con memoria dinámica

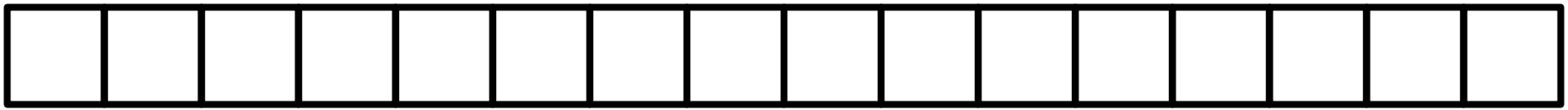
UNRN Universidad Nacional de Río Negro

fragmentación

UNRN Universidad Nacional de Río Negro

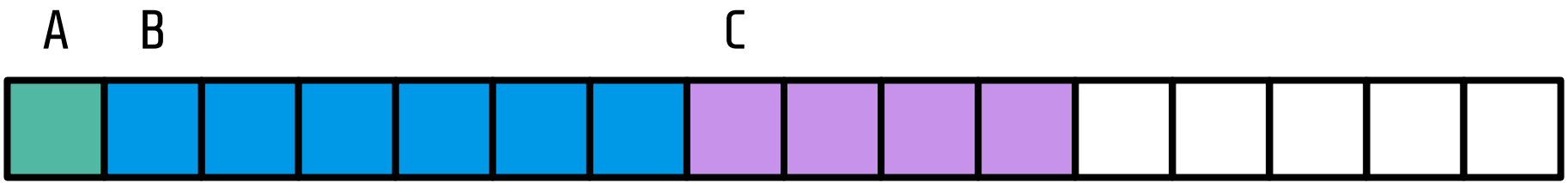
_

16 bloques libres



Sea esta toda la memoria disponible para `malloc`

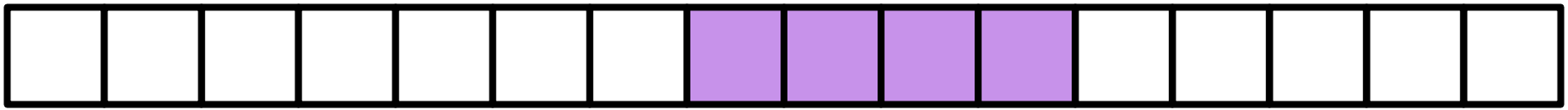
5 bloques libres



Hacemos tres pedidos de memoria (A, B y C)

12 bloques libres

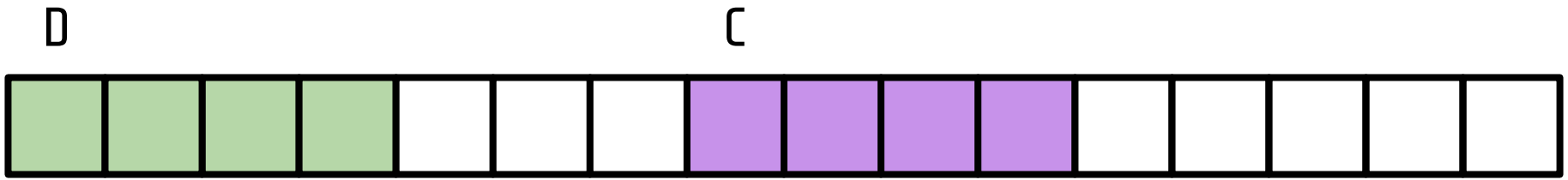
C



Liberamos A y B

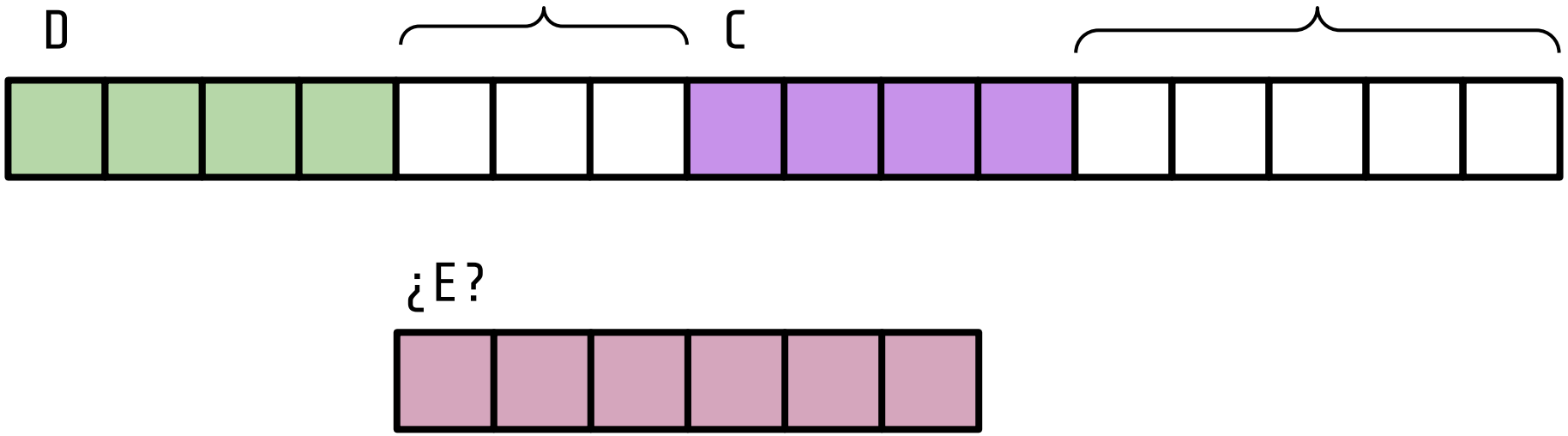
_

8 bloques libres



Pedimos un bloque D

8 bloques libres



Si necesitamos un bloque de 6, ¡no hay lugar!

Esto es un problema en programas que deben funcionar mucho tiempo


¿Preguntas?

¿Qué se puede hacer para reducir esto?

Pedir en bloque

Aunque el bloque sea estático, dinámico es 90% igual

```
typedef struct {
    char nombre[50];
    int edad;
} persona_t;

persona_t bloque[100] = {0};
int activo[100] = {};
```

Aunque el bloque sea estático, dinámico es 90% igual

```
typedef struct {
    char nombre[50];
    int edad;
} persona_t;

typedef struct {
  persona_t *bloque;
  int capacidad;
  int ultimo;
} almacen_t;
```

{ Pedir en bloque }

{ O reducir al mínimo la cantidad de `mallocs` }


¿Preguntas?

uso luego de free

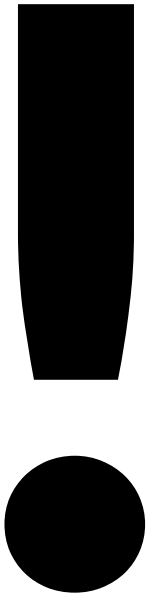
UNRN Universidad Nacional de Río Negro

Después de liberar un bloque de memoria, el puntero *sigue ahí*

Es lo mismo que retornar un puntero a una variable automática...



Punteros colgantes II

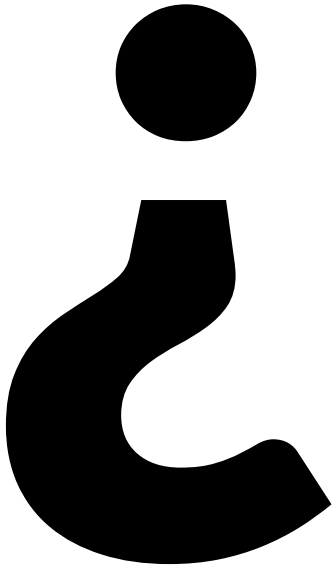


¡A la terminal!



{ **Para reducir estos problemas, NULL luego de liberar** }


¿Preguntas?



¿Es posible distinguir memoria dinámica de automática?



UNRN | Universidad Nacional de Río Negro

Si liberan algo no pedido dentro de una función...

Tal vez intenten liberar algo que no se puede liberar

¿En qué segmento de la memoria está cadena?

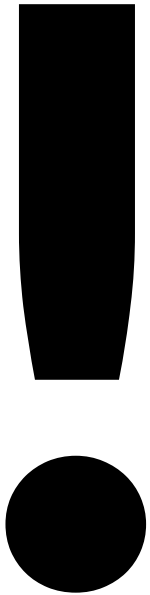
```
void funcion(char* cadena)
{
    free(cadena);
}
```

¿Que pasa acá?

```
int main()
{
    char automatica[] = “hola mundo”;
    funcion(automatica);
}
```



Comportamiento no definido



`free` solo puede liberar lo que `malloc` dá

{ Casi seguro un segfault }

Puede, potencialmente, romper la información que usa `malloc`

https://vulncat.fortify.com/en/detail?id=desc.controlflow.cpp.code_correctness_memory_free_on_stack_variable


¿Preguntas?

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar